

QoS based Priority Queuing in ToFino ASIC

Shashwat Patni

f20230390@goa.bits-pilani.ac.in

Introduction

This project is about figuring out how to Enable Priority Queuing on the TOFINO ASIC with P4 programmable Data plane .

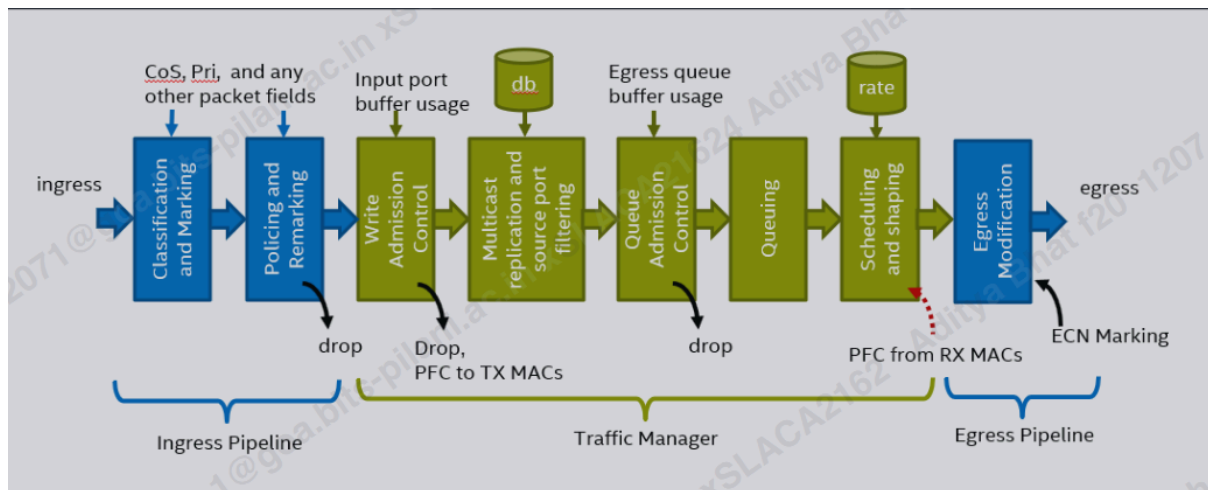
Broadly this can be classified into two steps:

- How to configure the Traffic Manager which is a fixed function component of the switch
- How to Use P4 to identify the QOS and Map it to the ICOS (architecture specific priority)

This midsem report covers the first point

Configuring the Traffic Manager

The Traffic Manager (TM) for the Tofino 1 architecture switch is a fixed function component which means it cannot be programmed like the Data plane can be. But for Tofino it is highly configurable. Which means we can assign priority to packets based on their headers.



Acronyms:

PPG: Priority Propagation Group

PFC: Priority Flow Control

ICOS: Ingress Class of Service

Getting Started:

After initializing ur P4 program , compiling it and running it on the switch daemon.

Inside the BfShell configure and enable the egress and ingress ports, this step is important. This is the step You do using bfsde -> UCLI -> enable ports

```
bf-sde> pm show
```

PORT	MAC	D_P	P/PT	SPEED	FEC	AN	KR	RDY	ADM	OPR	LPBK	FRAMES RX	FRAMES TX
29/0	3/0	24	0/24	100G	RS	Au	Au	---	DIS	DWN	NONE	0	0
32/0	2/0	16	0/16	100G	RS	Au	Au	---	DIS	DWN	NONE	0	0

Navigate back to the bf shell and start the BFRT_Python API

Navigate to BFRT.Tf1.TM which is how we configure the Traffic manager via the control plane. Use tab to autofill or guess the next values when navigating

Get the QID information for enabled Ports

Each TM Port maps the ingress metadata qid value to one of the queues

We need to get this mapping for our egress port (138 in case of this example)

```
bfrt.tf1.tm> port.cfg.get(dev_port=138)
Entry 0:
Entry key:
  dev_port      : 0x0000008A
Entry data:
  pg_id         : 0x02
  pg_port_nr    : 0x02
  port_queues_count : 0x08
  ingress_qid_map : [0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7, 0, 1, 2, 3, 4, 5, 6, 7]
  egress_qid_queues : [16, 17, 18, 19, 20, 21, 22, 23, 16, 17, 18, 19, 20, 21, 22, 23, 16, 17, 18, 19, 20, 21, 22, 23, 16, 17, 18, 19, 20, 21, 22, 23]
  copy_to_cpu    : False
  mirror_drop_destination : False
  mirror_drop_pg_queue : 0x00
  pkt_extraction_credits : 0x00000006
Out[44]: Entry for tf1.tm.port.cfg table.
```

If our p4 code assigns QID 4 to high traffic it will be mapped to queue 4 of the egress pipeline

Set Up Real Time Traffic (INGRESS)

We need to assign our ingress port to a PPG first. By default All are mapped to DPG (default priority group).

```
bfrt.tf2.tm.ppg.cfg.add_with_dev_port(
    pipe=0,
    ppg_id=1,
    dev_port=16
)
```

Dev_port **16** is our ingress port

ppg_id is the id for our new ppg (should be unique) remember it we will use it later.

Configuring the Ingress Buffer

Once mapped we start configuring the Buffer limits. For real time traffic we assign a guaranteed buffer.

```

bfrt.tf2.tm> ppg.cfg.get(pipe=0, ppg_id=1)
Entry 0:
Entry key:
    ppg_id                : 0x01
Entry data (action : dev_port):
    dev_port              : 0x00000010
    pfc_enable            : False
    pfc_skid_max_cells    : 0x00000000
    icos_0                : False
    icos_1                : False
    icos_2                : False
    icos_3                : False
    icos_4                : False
    icos_5                : False
    icos_6                : False
    icos_7                : False
    guaranteed_cells      : 0x00000000
    hysteresis_cells      : 0x00000080
    pool_id               : IG_APP_POOL_0
    pool_max_cells        : 0x00000000
    dynamic_baf           : DISABLE
    ppg_counter_id        : 0x00

```

Add the guaranteed buffer

```

bfrt.tf2.tm.ppg.cfg.mod_with_dev_port(
pipe=0, ppg_id=1,
guaranteed_cells=20)

```

Note: The guaranteed minimum buffer size should be chosen considering RX MTU size, otherwise packets with larger MTU will be dropped without shared space configured.

Map ICoS to PPG

map our example Real Time traffic with iCoS say 4 to the PPG associated with the ingress TM

```
bfrt.tf2.tm.ppg.cfg.mod_with_dev_port(  
    pipe=0,  
    ppg_id=1,  
    icos_4=True  
)
```

Setup Real Time Traffic (Egress)

Set up minimal buffering for out egress queue

```
bfrt.tf2.tm> queue.buffer.get(pipe=0, pg_id=3, pg_queue=4)  
Entry 0:  
Entry key:  
    pg_id          : 0x03  
    pg_queue       : 0x04  
Entry data (action : shared_pool):  
    guaranteed_cells : 0x00000024  
    hysteresis_cells : 0x00000020  
    tail_drop_enable : True  
    pool_id          : EG_APP_POOL_0  
    pool_max_cells   : 0x0000002B  
    dynamic_baf      : 80%
```

For the Real Time traffic class, we want to minimize buffer usage, so we decided to set:

- Guaranteed minimum buffer allocated as 20 cells (guaranteed_cells=20),
- Shared Pool limit to zero to disable its usage by this queue (pool_max_cells=0).

With BFRT TM API use action mod_with_buffer_only which will disable Shared Pool usage:

```
bfrrt.tf2.tm.queue.buffer.mod_with_buffer_only( pipe=0,  
pg_id=3,  
pg_queue=4,  
guaranteed_cells=20)
```

Enable Guaranteed Queue Rate for Real time Traffic (Traffic Shaping)

The Commands should be self explanatory of the functions. We do not limit the Max rates here.

```
bfrrt> tf2.tm.queue.sched_shaping.get(pipe=0, pg_id=3, pg_queue=4)  
Entry 0:  
Entry key:  
    pg_id          : 0x03  
    pg_queue       : 0x04  
Entry data:  
    unit           : BPS  
    provisioning   : UPPER  
    min_rate       : 0x00000000  
    min_burst_size : 0x00004000  
    max_rate       : 0x05F5E0FF  
    max_burst_size : 0x00004000
```

```
bfrrt.tf2.tm.queue.sched_shaping.mod(  
    pipe=0,  
    pg_id=3,  
    pg_queue=4,  
    min_rate=100000000  
)
```

```
bfrt.tf2.tm.queue.sched_cfg> get(pipe=0, pg_id=3, pg_queue=4)
Entry 0:
Entry key:
  pg_id          : 0x03
  pg_queue       : 0x04
Entry data:
  min_priority    : LOW
  min_rate_enable : False
  dwrr_weight     : 0x03FF
  max_priority    : LOW
  max_rate_enable : False
  advanced_flow_control : CREDIT
  scheduling_enable : True
  pg_ll_node      : 0x00
```

```
bfrt.tf2.tm.queue.sched_cfg.mod(
  pipe=0,
  pg_id=3,
  pg_queue=4,
  min_rate_enable=True,
  min_priority='HIGH'
)
```

Now all that is left is run the p4 code and generate the right Requirements for congestion